Numerical Mathematics II for Engineers

Christmas lecture: Linear solvers and multigrid, winter semester 2025/26

Dr. Peter Münch[†]

[†] Institute of Mathematics, Technical University of Berlin, Germany

December 18, 2025

Part 1:

Motivation: finite-element computations

Systems of linear equations

1. Poisson operator: Poisson problem
   - Original form:

$$\begin{aligned} -\Delta u &= f \quad \text{on } \Omega \\ u &= 0 \quad \text{on } \partial\Omega \end{aligned}$$

   - Discretization using FE: find $u_h \in V_h$:

$$a(u_h, \varphi_h) = (f, \varphi_h) \quad \forall \varphi \in V_h$$

   with

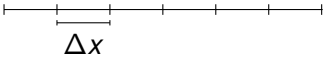
$$a(u_h, \varphi_h) = (\nabla u_h, \nabla \varphi_h).$$

   - Implementation (choice of basis, mesh, ...)

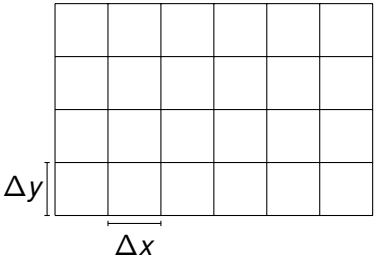
$$\boldsymbol{K}\boldsymbol{u} = \boldsymbol{F}$$

Cartesian meshes:

- 1D:



- 2D:



Systems of linear equations (cont.)

② mass matrix: L2 projection, find $u_h \in V_h$:

$$(u_h, \varphi_h) = (f, \varphi_h) \quad \forall \varphi \in V_h \quad \rightarrow \quad \boldsymbol{M}\mathbf{u} = \boldsymbol{F}$$

... also: explicit time stepping $\rightarrow$ mass lumping/collocation methods

③ Helmholtz operator: implicit time stepping, e.g., backwards Euler method

$$(\boldsymbol{M} + \Delta t \boldsymbol{K})\boldsymbol{u}_{i+1} = \boldsymbol{M}\mathbf{u_i}$$

observation: $\boldsymbol{M}$ dominates for $\Delta t \rightarrow 0$ and $\boldsymbol{K}$ dominates for $\Delta t \rightarrow \infty$

In the following, we concentrate on (1D/2D) Poisson operator $\boldsymbol{K}$ and mass matrix $\boldsymbol{M}$.

Goal of the lecture: how to solve $\boldsymbol{K}\boldsymbol{u} = \boldsymbol{F}$ (hard) and $\boldsymbol{M}\boldsymbol{u} = \boldsymbol{F}$ (easy)?

Table of contents

Goal of the lecture: how to solve $\boldsymbol{K}\boldsymbol{u} = \boldsymbol{F}$ (hard) and $\boldsymbol{M}\boldsymbol{u} = \boldsymbol{F}$ (easy)?

Table of contents:

(1) Matrix properties: eigenvalues and conditioning $\kappa = O(N^2)$

(2) Basis linear solvers: revision of concepts from NLA $N_{\text{it}} = O(N^2)$

(3) Two-level algorithm: **new** $N_{\text{it}} = O(1)$

(4) Multigrid: **new** $N_{\text{work}} = O(N)$

Part 2:

Matrix properties

Properties in 1D

Example: 1D domain [0, 1] with homogeneous DBC at left and right end and equidistant mesh with $\Delta x = 1/(N+1)$

Stiffness matrix $K \in R^{N\times N}$:

$$K = \frac{1}{\Delta x}\begin{bmatrix} 2 & -1 & & & 0 \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ 0 & & & -1 & 2 \end{bmatrix}.$$

Mass matrix $M \in R^{N\times N}$:

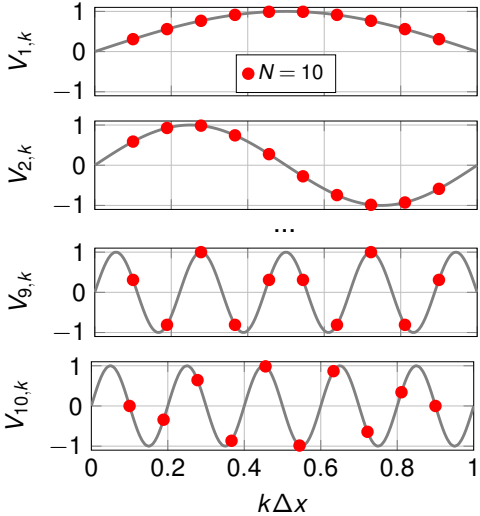
$$M = \frac{\Delta x}{6}\begin{bmatrix} 4 & 1 & & & 0 \\ 1 & 4 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & 4 & 1 \\ 0 & & & 1 & 4 \end{bmatrix}.$$

<u>Properties:</u>

- symmetric positive-definite (SPD)
- diagonally dominant
- sparse (tridiagonal) $\rightarrow$ matrix-vector product $O(N)$
  $\rightarrow$ easy to solve with $O(N)$ with Thomas algorithm

Properties in 1D: eigenvalues/eigenvectors

low-frequency mode



high-frequency mode

The jth eigenvector of K and M is given by:

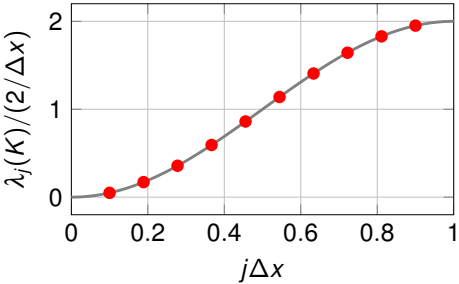
$$V_{j,k} = \sin(\pi k h j), \quad 1 \leq k \leq N.$$

Proof: Apply eigenvector to any row of the matrix.

Properties in 1D: Eigenvalues/eigenvectors (cont.)

The jth eigenvalue of K and M is given by:

$$\lambda_j(K) = \frac{2}{\Delta x}(1 - \cos(\pi k \Delta x))$$

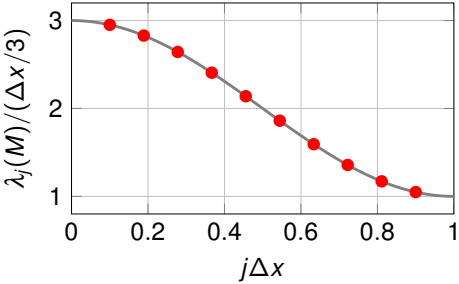


with $|\lambda_k(K)| \in [\pi^2 \Delta x,\ \frac{4}{\Delta x}]$, which implies

$$\boxed{\kappa(K) = \mathcal{O}(1/\Delta x^2)}$$

Taylor expansion: $1 - \cos(\pi x) \sim \frac{\pi}{2} x^2$

$$\lambda_j(M) = \frac{\Delta x}{3}(2 + \cos(\pi k \Delta x))$$



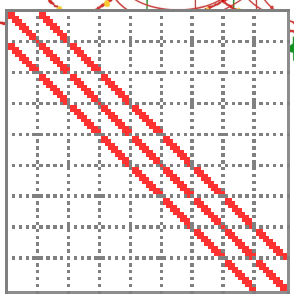
with $|\lambda_k(M)| \in [\frac{\Delta x}{3},\ \Delta x]$, which implies

$$\boxed{\kappa(M) \to 3 = \mathcal{O}(1)}$$

Properties in 2D

For Cartesian 2D meshes:

- the matrices are sparse and block-diagonal
- eigenvalues are given by the ones of 1D problems



Note: the generalized eigendecomposition of 2D K:

$$K = M_1 \otimes K_0 + K_1 \otimes M_0 = T_1 \otimes T_0 (\Lambda_1 \otimes I_0 + I_1 \otimes \Lambda_0) T_1^\top \otimes T_0^\top,$$

where $T \in \mathbb{R}^{n \times n}$ and $\Lambda \in \mathbb{R}^n$ are given by generalized eigendecomposition $K_i T_i = M_i T_i \Lambda_i$.

Summary

Checklist:

- ☑ the finite-element discretization leads to sparse matrices
- ☑ symmetric, positive-definite, diagonally dominant
- ☑ eigenvalues
- ☑ conditioning: $\kappa(K) = O(h^{-2})$

We need linear solvers!

Part 3:

(Basic) Linear solvers

Direct solvers

Solve ($A \in \mathbb{R}^{N \times N}$, $x, b \in \mathbb{R}^N$)

$$Ax = b$$

by:

- step 1: decomposing $A = LU$ ▷ $O(N^3)$
- step 2: forward/backward substitution: ▷ $O(N^2)$

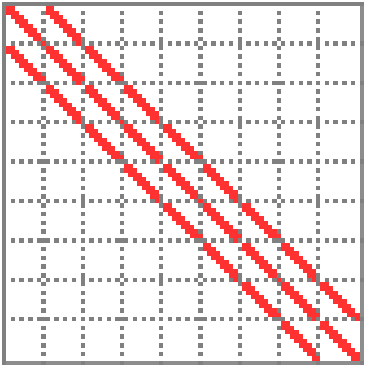
$$Ly = b,$$
$$Ux = y.$$

Advantage: reuse L and U for different b.

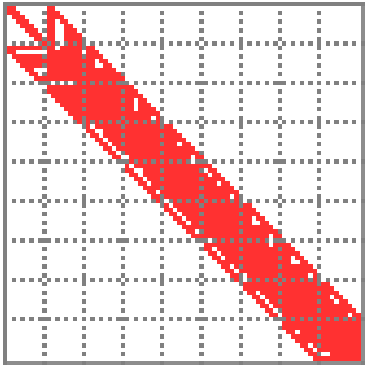
Direct solvers (cont.)

LU factorization can be applied to sparse matrices: $O(Nb) \overset{2D}{\approx} O(N^{3/2})$

A (NNZ: 625)



$L + U$ (NNZ: 1409)



Observation: fill-in in L/U. It is related to the bandwidth b of A, which is related to the ordering of the degrees of freedom. Strategy to reduce fill-in: Cuthill—McKee algorithm.
$\Rightarrow$ increased memory consumption and costs, making direct solvers infeasible in 3D.

Iterative solvers: relaxation methods

(Over-)relaxation schemes have the following form:

$$x_{i+1} = (1-\omega)x_i + \omega\tilde{x}_{i+1}.$$

Let's choose $\tilde{x}_{i+1} = x_i + r_i = x_i + (b - Ax_i) = (I - A)x_i + b$.

We get

- residual notation

$$x_{i+1} = x_i + \omega r_i$$

- notation of analysis

$$x_{i+1} = (I - \omega A)x_i + \omega b$$

Iterative solvers: relaxation methods (cont.)

We define the error:

$$e_{i+1} = x_{i+1} - x.$$

We get

$$e_{i+1} = (I - \omega A)x_i - \omega b - x + \omega(\underbrace{b - Ax}_{=0}) = (I - \omega A)e_i = (I - \omega A)^{i+1} e_0.$$

We call $(I - \omega A)$ the iteration matrix C.

Iterative solvers: relaxation methods (cont.)

Theorem: The iteration scheme converges if all $|\lambda_k(C)| < 1$ with convergence speed $\rho = \max_k |\lambda_k(C)|$.

Proof:

- consider eigenvalues μ_k and eigenvectors v_k of iteration matrix C:

$$C\left(\sum_k \alpha_k v_k\right) = \sum_k \alpha_k C v_k = \sum_k \mu_k \alpha_k v_k$$

- write error as combination of eigenvectors: $e^{(0)} = \sum_k \alpha_k v_k$:

$$e_{i+1} = Ce_i = C^{i+1} e_0 = \sum_k \mu_k^{i+1} \alpha_k v_k$$

$\square$

Iterative solvers: relaxation methods (cont.)

Following statements can be made regarding $(I - \omega A)$:

- eigenvalues of $(I - \omega A)$:

$$\lambda_k(I - \omega A) = 1 - \omega \lambda_k(A)$$

- max. eigenvalue of $(I - \omega A)$:

$$\rho(I - \omega A) = \max_k \{1 - \omega \lambda_k(A)\}$$

- convergence for

$$0 < \omega < \omega_{\max}, \text{ w. } \omega_{\max} = \frac{2}{\lambda_{\max}(A)}$$

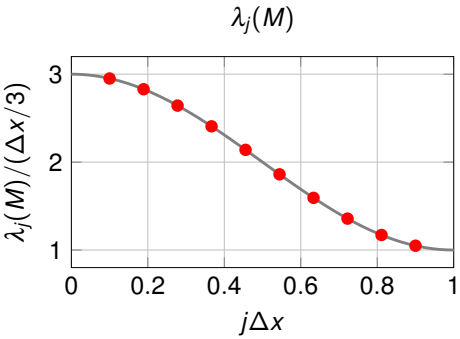
- optimal convergence for

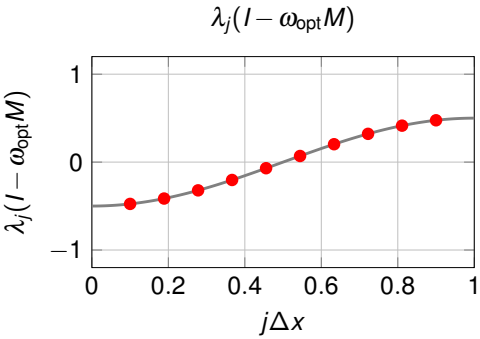
$$\omega_{\text{opt}} = \frac{2}{\lambda_{\min}(A) + \lambda_{\max}(A)}$$

Iterative solvers: relaxation methods (cont.)

Example: mass matrix:

$\lambda_{\min} = \frac{\Delta x}{3}$ and $\lambda_{\max} = \Delta x$ give $\omega_{\text{opt}} = \frac{2}{\lambda_{\min}+\lambda_{\max}} = \frac{3}{2}\frac{1}{\Delta x} = \text{diag}_i(M)^{-1}$



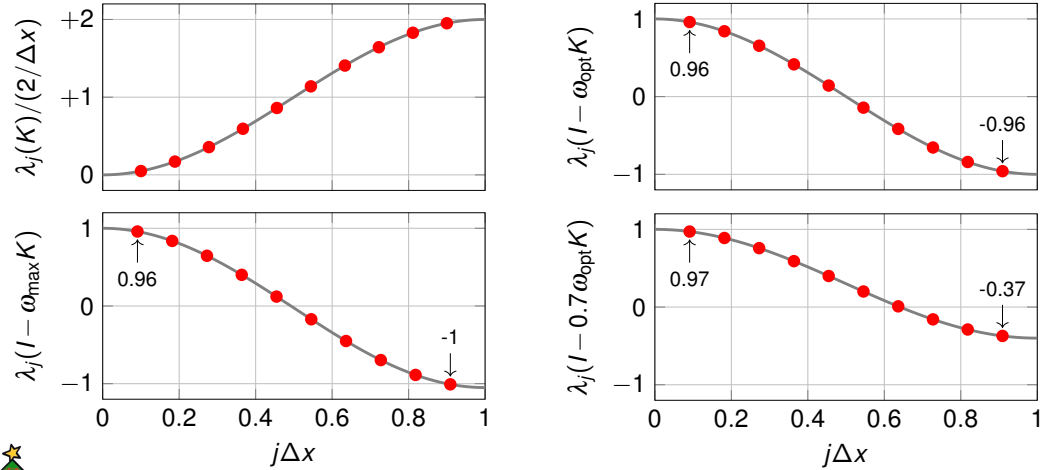


Observation: $\rho = 0.5$ *independent* of h. 10 iterations to reduce the error by $\times 1000$!

Iterative solvers: relaxation methods (cont.)

▷ for $h \to 0$: $\omega_{\max} = \omega_{\text{opt}} = \text{diag}_i(K)^{-1}$

Example: Poisson operator:



Observation: low frequencies are damped very slowly; high frequencies are damped for $\omega < \omega_{\max}$. We say: high-frequency errors are well smoothed. The scheme is a smoother.

Iterative solvers: preconditioning

For our case (constant diagonal entries), $x_{i+1} = x_i + \omega_{\text{opt}}(b - Ax_i)$ is equivalent to

$$x_{i+1} = x_i + P^{-1}(b - Ax_i),$$

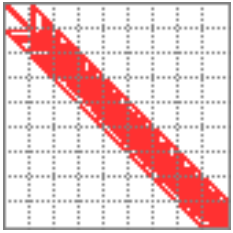
where $P = \text{diag}(A)$, implying that it is the Jacobi method.

P is a preconditioner.[1] Others:

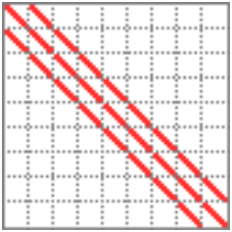
- Gauss-Seidel method
- SSOR
- ILU (incomplete LU)
- multigrid (later in this lecture!)

ILU: no fill-in under a certain tolerance is allowed

$L + U$ (NNZ: 1409)



$\tilde{L} + \tilde{U}$ (NNZ: 625)

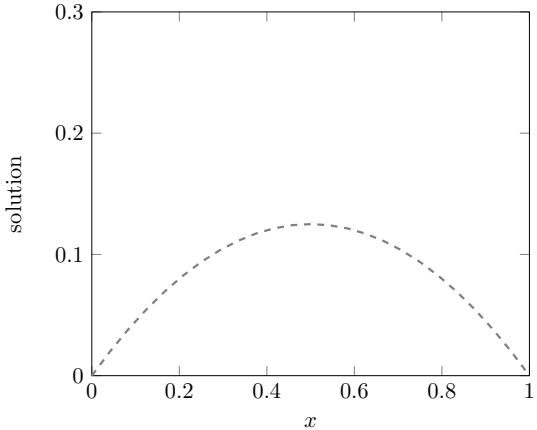


[1] Requirements: • P^{-1} should be easy to apply to a vector; • improved conditioning $\rho(P^{-1}A) \ll \rho(A)$.

Iterative solvers: example

Example: 1D Poisson problem

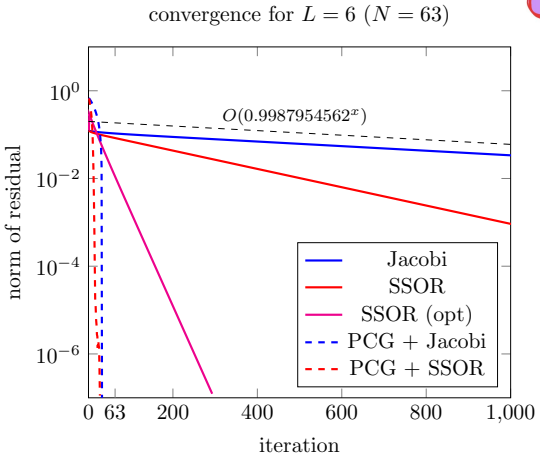
- on [0, 1]
- mesh with 1 cell, L-times refined
- with constant rhs
- solve until $|r_i|/|r_0| < 10^{-6}$
- smooth initial guess: $x_0 = 0$



Iterative solvers: example (cont.)

We compare:

- (PCG+) Jacobi
- (PCG+) SSOR
- SSOR vs. SSOR (opt)



Sanity check for Jacobi ($L = 6$):

$$\lambda_j = cos(j\Delta x\pi) \quad \Rightarrow \quad \rho = \lambda_1 = cos(\Delta x\pi) = 0.9987954562$$

Iterative solvers: example (cont.)

L	Jacobi	SSOR	SSOR (opt.)	PCG + Jacobi	PCG + SSOR
2	41	17	16	2	3
3	175	52	34	4	6
4	710	186	71	8	10
5	-	721	144	16	16
6	-	-	293	32	27
7	-	-	599	64	49
8	-	-	-	128	66
9	-	-	-	256	107
N_{it}	$O(N^2)$	$O(N^2)$	$O(N)$	$O(N)$	$O(N)$
N_{work}	$O(N^3)$	$O(N^3)$	$O(N^2)$	$O(N^2)$	$O(N^2)$

Sanity check for Jacobi:

$$\lambda_j = cos(j\Delta x\pi) \quad \Rightarrow \quad \rho = \lambda_1 = 1 - \frac{\pi^2}{2}\Delta x^2 \quad \Rightarrow \quad N_{\text{it}} = O(N^2)$$

Summary

Checklist:

- ☑ For large problem sizes, direct solvers are not feasible $\rightarrow$ fill-in.
- ☑ The efficiency of iterative solvers (relaxation methods & Krylov methods) heavily depends on $\kappa(A)$.
- ☑ Poisson operator: $\kappa(A) \sim 1/h^2$
- ☑ We need (1) <u>a solver that can handle ill-conditioned matrices</u> (i.e., high $\kappa(A)$) or (2) a preconditioner that improves the conditioning of the problem (i.e., $\kappa(P^{-1}A) \ll \kappa(A)$).

Solution: two-level algorithm!

Part 4:

Two-level algorithm

Idea

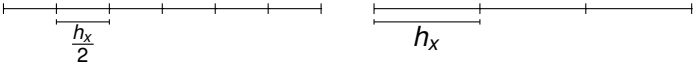
Observation:

- Smoothers damp high-frequency errors well.
- Low-frequency errors become high-frequency errors on coarser grids.

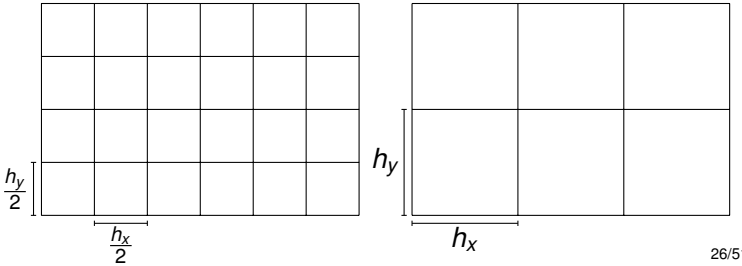
Idea: use smoother on fine grid and apply (direct) solver on coarse grid.

Examples: nested grids

- 1D:



- 2D:



Two-level algorithm

Algorithm [two-level algorithm]: Solve the system of linear equations $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$:

- presmoothing:

$$\boldsymbol{x} \leftarrow S(\boldsymbol{x}, \boldsymbol{b})$$

- coarse-grid correction:

$$\boldsymbol{r} \leftarrow \boldsymbol{b} - A(\boldsymbol{x}), \quad \boldsymbol{b}_c \leftarrow R(\boldsymbol{r}), \quad \boldsymbol{v} \leftarrow A_c^{-1}\boldsymbol{b}_c, \quad \boldsymbol{x} \leftarrow \boldsymbol{x} + P(\boldsymbol{v})$$

- postsmoothing:

$$\boldsymbol{x} \leftarrow S(\boldsymbol{x}, \boldsymbol{b})$$

Building blocks: (pre/post) smoother S,
intergrid transfer (restriction R/prolongation P), coarse-grid solver A_c^{-1}

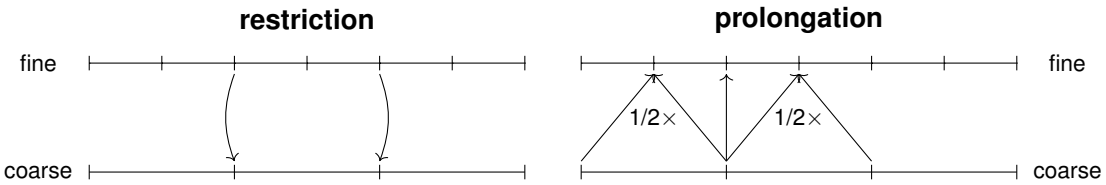
Example

L	Jacobi	SSOR	SSOR (opt.)	Two-level algorithm
2	41	17	16	6
3	175	52	34	6
4	710	186	71	5
5	-	721	144	5
6	-	-	293	5
7	-	-	599	5
8	-	-	-	5
9	-	-	-	5
N_{it}	$O(N^2)$	$O(N^2)$	$O(N)$	$O(1)$
N_{work}	$O(N^3)$	$O(N^3)$	$O(N^2)$	?

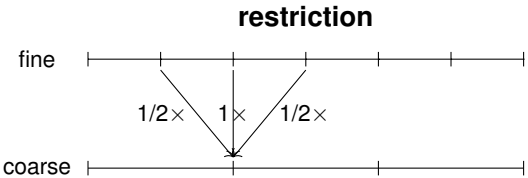
Observation: constant number of iterations! What are the costs?

Building blocks: Intergrid transfer operators

1D intergrid transfer operators:



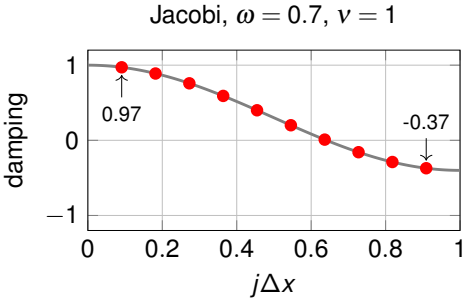
Warning: in the FEM context, one often chooses $R = P^\top$:

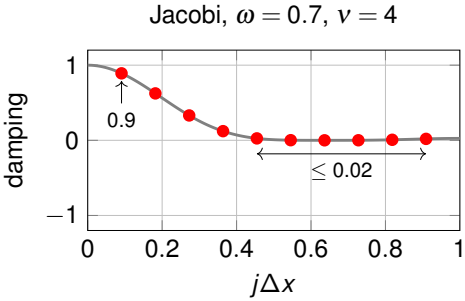


Similar in 2D. See homework!

Building blocks: Smoothers

- use *simple* smoothers like Jacobi (or Gauss-Seidel)
- choose relaxation parameter so that high-frequency errors are damped!
- optional: apply smoother multiple times ($\times \nu$)





Building blocks: Coarse-grid operator/solver

Coarse-grid operator A_c:

- Galerkin projection

$$A_c = RAP$$

- rediscretization: evaluate operator with coarser mesh size

Coarse-grid solver A_c^{-1}:

- direct solver and iterative solver $\rightarrow \mathcal{O}(N_c^2) = \mathcal{O}(N^2) \rightarrow$ expensive!
- call algorithm recursively $\rightarrow$ multigrid algorithm (later)

Two-level algorithm: convergence analysis

The iteration matrix of the two-level matrix (simplification: only presmoothing) is

$$S_{tl} = (A_f^{-1} - PA_c^{-1}R)A_f S_{sm}^{\nu}.$$

Goal of the convergence analysis is to show that $||S_{tl}|| \leq \rho \leq 1$, where ρ is independent of h. For this purpose, we look at

$$||S_{tl}|| \leq \underbrace{||(A_f^{-1} - PA_c^{-1}R)||}_{①} \underbrace{||A_f S_{sm}^{\nu}||}_{②} \leq C_a \eta(\nu) \leq \rho.$$

Approximation property

① $||(A_f^{-1} - PA_c^{-1}R)|| \leq C_a h^{\alpha}$

Smoothing property

② $||A_f S_{sm}^{\nu}|| \leq \eta(\nu) h^{-\alpha}$

For more details, see V. John 2013.

Summary

Checklist:

- ☑ Key building blocks of two-level algorithms: (pre/post) smoother, intergrid transfer (restriction/prolongation), coarse-grid solver.
- ☑ Mesh-independent convergence: smoothing + approximation property.
- ☑ Coarse-grid solver might be expensive!

Solution: the multigrid algorithm!

Part 5:

Multigrid

Multigrid algorithm

Idea: apply two-level algorithm recursively.

Algorithm [multigrid algorithm]: Solve the system of linear equations $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$:

- **if** $l = 0$:
  - coarse-grid solver (with initial condition $\boldsymbol{x}$)

$$\boldsymbol{x} \leftarrow A_0^{-1}\boldsymbol{b}$$

- **else**:
  - presmoothing:

$$\boldsymbol{x} \leftarrow S_l(\boldsymbol{x}, \boldsymbol{b})$$

  - coarse-grid correction:

$$\boldsymbol{r} \leftarrow \boldsymbol{b} - A_l(\boldsymbol{x}), \quad \boldsymbol{b}_c \leftarrow R_l(\boldsymbol{r}), \quad \boldsymbol{v} \leftarrow 0, \quad \underbrace{\boldsymbol{v} \leftarrow \textbf{multigrid}(\boldsymbol{v}, \boldsymbol{b}_c, l-1)}_{R \text{ times}}, \quad \boldsymbol{x} \leftarrow \boldsymbol{x} + P_l(\boldsymbol{v})$$

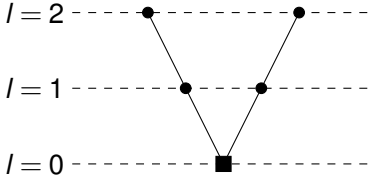
  - postsmoothing:

$$\boldsymbol{x} \leftarrow S_l(\boldsymbol{x}, \boldsymbol{b})$$

V- vs. W-cycle

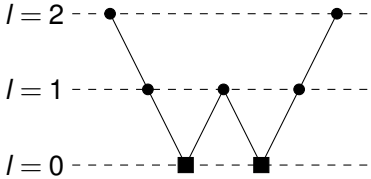
Structure of MG method is determined by parameter R. Common choices: $R \in \{1, 2\}$.

V-cycle ($R = 1$):



+ efficient
- not always stable

W-cycle ($R = 2$):



+ robust
- expensive

with • = smoothing, ■ = coarse-grid solver, and — = transfer

Example

L	**Two-level algorithm**	**V-cycle**	**W-cycle**
2	6	6	6
3	6	6	6
4	5	6	5
5	5	6	5
6	5	6	5
7	5	6	5
8	5	7	5
9	5	7	5
N_{it}	$O(1)$	$O(1)$	$O(1)$
N_{work}	?	?	?

Multigrid algorithm: convergence

For details, see V. John 2013.

Multigrid algorithm: complexity

The cost of one cycle of multigrid is given by the sum of the cost of each level:

$$\mathcal{C} = \sum \mathcal{C}_l = \mathcal{C}_L \sum (\mathcal{C}_l / \mathcal{C}_L) = O(N).$$

Proof:

1 The cost of the finest level is given by:

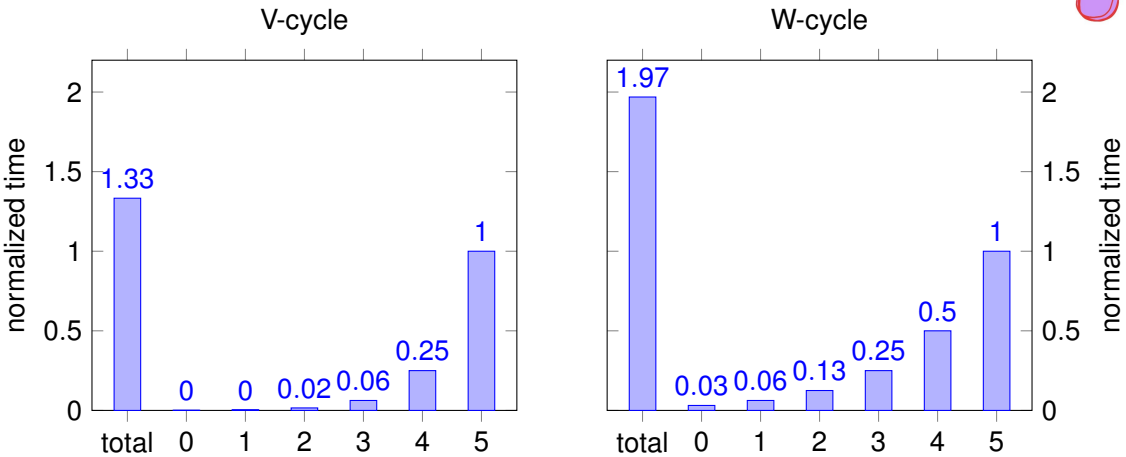
$$\mathcal{C}_L = \nu_{pre}\mathcal{C}^L_{pre} + \mathcal{C}^L_{rest} + \mathcal{C}^L_{pro} + \nu_{post}\mathcal{C}^L_{post} = O(N).$$

2 The cost of a coarser level can be related to the finest level by a the reduction of the cost per level and by b the number of executions:

$$\text{V-cycle: } \sum \frac{\mathcal{C}_l}{\mathcal{C}_L} = \sum_i \frac{1}{2^{d\cdot i}} \leq \frac{1}{1-\frac{1}{2^d}} \overset{\text{2D}}{=} \frac{4}{3},$$

$$\text{W-cycle: } \sum \frac{\mathcal{C}_l}{\mathcal{C}_L} = \sum_i \frac{2^i}{2^{d\cdot i}} = \sum_i \frac{1}{2^{(d-1)\cdot i}} \overset{\text{2D, 3D}}{=} \frac{1}{1-\frac{1}{2^{d-1}}} \overset{\text{2D}}{=} 2.$$

Multigrid algorithm: complexity (cont.)



Observation: one cycle of W-cycle is 50% more expensive!

Example

L	Two-level algorithm	V-cycle	W-cycle
2	6	6	6
3	6	6	6
4	5	6	5
5	5	6	5
6	5	6	5
7	5	6	5
8	5	7	5
9	5	7	5
N_{it}	$O(1)$	$O(1)$	$O(1)$
N_{work}	$\sim O(N^2)$	$O(N)$	$O(N)$ (2D, 3D)

Observation: Multigrid needs constant work but introduces a lot of overhead.
Rule of thumbs: Multigrid is only useful if Jacobi needs at least 50 linear iterations.

Homework

Programming: multigrid for 2D finite difference method

TU Berlin
Institute of Mathematics
Prof. Dr. Sandra May
Dr. Peter Münch

Numerical methods for PDEs — Winter 2024/25

14. Homework set

Deadline: Wednesday, Feb 12 2025

Problem 14.1: Programming: 2d multigrid

In the lecture, you learned about the (geometric) multigrid algorithm to solve systems of linear equations originating from elliptic PDEs. The basic algorithm of a V-cycle is shown in Algorithm 1. In this task, you will implement the algorithm yourself. However, in order to be able to do so, we need to discuss the details of the relevant building blocks.

Problem

Solve the Poisson equation

$$-\Delta u = f(\vec{x}) \quad \text{in } \Omega, \tag{1a}$$
$$u = 0 \quad \text{on } \Gamma, \tag{1b}$$

with the finite difference method using a 5-point stencil. The domain Ω is given as a rectangle $[0, a] \times [0, b]$ and the mesh $\mathcal{T}^{(L)}$ is obtained by refining L times a Cartesian (coarse) grid $\mathcal{T}^{(0)}$. The mesh-refinement levels naturally give the multigrid levels, as shown in Figure 1.

Algorithm 1: Multigrid V-cycle called recursively on each level l to solve $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, where $\boldsymbol{x}$ is also used as initial condition. The algorithm is called with $\boldsymbol{x} \leftarrow \texttt{VCycle}(L, \boldsymbol{x}, \boldsymbol{f})$ if used as solver and with $\boldsymbol{z} \leftarrow \texttt{VCycle}(L, \boldsymbol{0}, \boldsymbol{r})$ if used as preconditioner of, e.g., CG. The smoothers are each m times. As coarse-grid solver, we apply a smoother n times.

```
if l = 0 then
    for i ← 1 to n do
        x ← S^(0)(x, b) ;                          /* coarse-grid solver: smooth n times */
    return x
else
    for i ← 1 to m do
        x ← S^(l)(x, b) ;                          /* presmoothing m-times */
    r ← b − A^(l) x ;                              /* compute residual */
    f̄ ← R_(l−1)^(l) r ;                            /* restrict residual */
    z̄ ← VCycle(l − 1, 0, f̄) ;                      /* recursion */
    x ← x + P_(l−1)^(l) (z̄) ;                      /* prolongation */
    for i ← 1 to m do
        x ← S^(l)(x, b) ;                          /* postsmoothing m times */
    return x
```

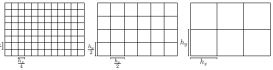



Figure 1: Sequence of meshes $\mathcal{T}^{(l)}$ obtained by refining a coarse grid containing of 3×2 cells.

Building blocks

In the following, we discuss intergrid transfer operators, level operators, smoothers, and the coarse-grid solver. We refer to these as operators, since we will never explicitly compute the corresponding matrices.

Intergrid transfer operators

The *restriction operator* $R_{(l)}^{(l-1)}$ is given by injection:

$$r_{i,j}^{(l)} = r_{2i,2j}^{(l+1)} \tag{2}$$

and the *prolongation operator* $P_{(l-1)}^{(l)}$ via interpolation from the coarse points to the fine points. Depending on the position of the fine points (coinciding with coarse points, with coarse horizontal or vertical lines, or with the center of coarse cells), one gets different interpolation formulae:

$$x_{2i,2j}^{(l+1)} = x_{i,j}^{(l)} \tag{3a}$$
$$x_{2i+1,2j}^{(l+1)} = (x_{i,j}^{(l)} + x_{i+1,j}^{(l)})/2, \tag{3b}$$
$$x_{2i,2j+1}^{(l+1)} = (x_{i,j}^{(l)} + x_{i,j+1}^{(l)})/2, \tag{3c}$$
$$x_{2i+1,2j+1}^{(l+1)} = (x_{i,j}^{(l)} + x_{i+1,j}^{(l)} + x_{i,j+1}^{(l)} + x_{i+1,j+1}^{(l)})/4. \tag{3d}$$

Figure 2 gives a visualization of the performed operations.

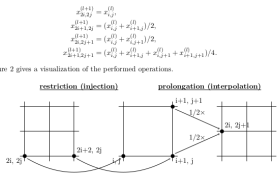


Figure 2: Visualization of restriction and prolongation.

Summary

Checklist:

- ☑ Multigrid methods call the two-level algorithm recursively.
- ☑ If the recursion is called once or twice per level, we get a V-cycle or a W-cycle respectively.
- ☑ Multigrid methods can converge in $O(1)$ iterations.
- ☑ Each iteration has the cost $O(N)$, implying the total cost of $O(N)$!

Part 6:

Multigrid: outlook

Multigrid as preconditioner

E.g., Richardson algorithm:

$$x^{(k+1)} = x^{(k)} + \omega P^{-1} \underbrace{(b - Ax^{(k)})}_{r^k}.$$

Call multigrid with

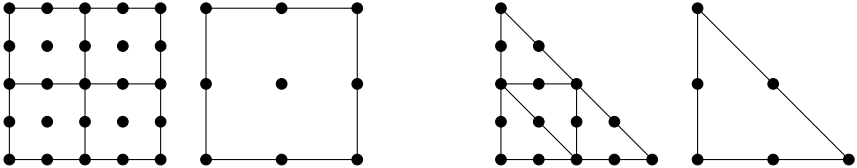
$$\textbf{multigrid}(0, r^k, L).$$

In practice, we would use conjugate gradient (CG) method (or generalized minimal residual, GMRES, method for non-symmetric matrices).

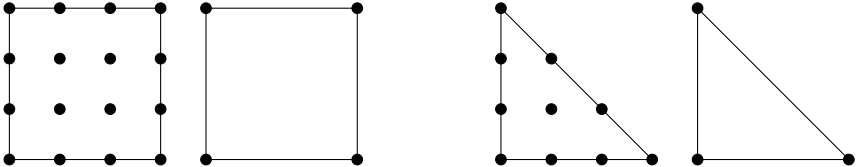
L	**V-cycle**	**PCG + V-cycle**
2	6	2
3	6	3
4	6	4
5	6	4
6	6	4
7	6	4
8	7	4
9	7	4
N_{it}	$O(1)$	$O(1)$
N_{work}	$O(N)$	$O(N)$

Definition of levels

- h-multigrid: coarse mesh by combining cells (Q_2/P_2)



- p-multigrid: coarse polynomial degree ($Q_3 \to Q_1/P_3 \to P_1$)



- non-nested levels
- given a matrix A, determine algebraically coarser grids $\to$ algebraic multigrid (AMG)

Multigrid for (Navier–)Stokes operator

Incompressible flow is described by the incompressible Navier–Stokes equations:

$$\rho\left(\frac{\partial \mathbf{u}}{\partial t}+(u\cdot\nabla)\mathbf{u}\right)=-\nabla p+\nabla\cdot\tau_\mu+\rho\mathbf{g}+\mathbf{f}$$

$$\nabla\cdot\mathbf{u}=0$$

This is a non-linear set of equations. We can use FEM and Newton method. The Jacobian has a saddle-point-problem structure. Its block LU factorization is given by (with Schur complement: $S := -CA^{-1}B$):

$$J=\begin{bmatrix} A & B \\ C & 0 \end{bmatrix}=\underbrace{\begin{bmatrix} I & \\ CA^{-1} & I \end{bmatrix}}_{L}\underbrace{\begin{bmatrix} I & \\ & S \end{bmatrix}\begin{bmatrix} A & B \\ & I \end{bmatrix}}_{U}.$$

Approximate A^{-1} with one V-cycle of multigrid and S with the inverse of a scaled mass matrix.

Other examples: computational solid mechanics.

(Open-source) Software

There are numerous open-source implementations of multigrid available. Although the idea of multigrid seems to be easy, an efficient parallel implementation is challenging; consider, e.g., the Galerkin projection *RAP*, which is a sparse matrix-matrix-matrix product.

Examples:

- hypre: BoomerAMG → AMG
- Trilinos: ML, MueLu → AMG
- PyAMG in Python → AMG
- deal.II:[2] geometric and *p*-multigrid for locally refined meshes

[2] developed also at Chair of Numerical Methods for Partial Differential Equations, TUB

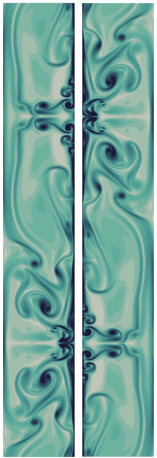
Advanced numerical methods for computational fluid dynamics

New: practical course, 10 credits, starting summer semester 2026
Focus: **(stabilized) incompressible Navier–Stokes equations:**

$$
\begin{aligned}
&(\partial_t \mathbf{u},\ \mathbf{v}) + (\mathbf{u}\cdot\nabla\mathbf{u},\ \mathbf{v}) - (p,\ \nabla\cdot\mathbf{v}) + (\nu\varepsilon(\mathbf{u}),\ \varepsilon(\mathbf{v})) \\
&\qquad + \delta_1\left(\partial_t\mathbf{u} + \mathbf{u}\cdot\nabla\mathbf{u} + \nabla p - \nu\Delta\mathbf{u},\ \mathbf{u}\cdot\nabla\mathbf{v}\right) + \delta_2\left(\nabla\cdot\mathbf{u},\ \nabla\cdot\mathbf{v}\right) = 0, \\
&(\nabla\cdot\mathbf{u},\ q) + \delta_1\left(\partial_t\mathbf{u} + \mathbf{u}\cdot\nabla\mathbf{u} + \nabla p - \nu\Delta\mathbf{u},\ \nabla q\right) = 0
\end{aligned}
$$

Solve with BDF2/ST and Newton–Krylov method with Jacobian:

$$
J = \begin{bmatrix} A & B^T \\ C & D \end{bmatrix} \quad \rightarrow \quad J' = \begin{bmatrix} A & B^T \\ 0 & S \end{bmatrix} \quad \rightarrow \quad \text{(monolithic) multigrid}
$$



Topics:
– C++/Linux
– basics of FEM
– Navier–Stokes eq.
– adaptivity
– open-source: deal.II

Format:
– lectures
– exercise sessions
– group work
– project

deal.II

(ACM/SIAM CSE 2025 Winner)

Fröhliche Weihnachten und einen guten Rutsch!

Happy Christmas and a happy new year!

¡Feliz Navidad y Próspero Año Nuevo!

Boldog Karácsonyt és BÚÉK!!!

References

Introduction

- Briggs, W.L., Henson, V.E. and McCormick, S.F., 2000. A multigrid tutorial. SIAM.
- John, V., 2013. Multigrid methods. Tech. report.

Current research

- N. Margenberg, M. Bause, P. Munch, An hp multigrid approach for tensor-product space-time finite element discretizations of the Stokes equations, SIAM SISC, 2025
- P. Munch, I. Dravins, M. Kronbichler, M. Neytcheva, Stage-parallel fully implicit Runge-Kutta implementations with optimal multilevel preconditioners at the scaling limit, SIAM SISC, 2024.
- P. Munch, T. Heister, L. Prieto Saavedra, M. Kronbichler, Efficient distributed matrix-free multigrid methods on locally refined meshes for FEM computations, ACM TOPC, 2023.